# Lab 1: Variables & Functions, Control

**lab01.zip (lab01.zip)**

*Due by 11:59pm on Tuesday, September 1.*

## Starter Files

Download lab01.zip (lab01.zip). Inside the archive, you will find starter files for the questions in this lab, along with a copy of the Ok (ok) autograder.

Additionally, please fill out this survey (https://forms.gle/BP3tcRgqPhHq4Ymf9) with any issues you might have faced in Lab 0 Python installation or if you used the Windows automated installer.

**For quickly generating ok commands, you can now use the ok command generator (https://links.cs61a.org/ok-help).**

# Quick Logistics Review

Using Python

Using OK

# Topics

Consult this section if you need a refresher on the material for this lab. It's okay to skip directly to the questions and refer back here should you get stuck.

Division

Functions

Control

Error Messages

# Required Questions

## What Would Python Display? (Part 1)

### Q1: WWPD: Control

Use Ok to test your knowledge with the following "What Would Python Display?" questions:

```
python3 ok -q control -u
```

**Hint**: Make sure your `while` loop conditions eventually evaluate to a false value, or they'll never stop! Typing `Ctrl-C` will stop infinite loops in the interpreter.

```
>>> def xk(c, d):
...     if c == 4:
...         return 6
...     elif d >= 4:
...         return 6 + 7 + c
...     else:
...         return 25
>>> xk(10, 10)
______

>>> xk(10, 6)
______

>>> xk(4, 6)
______

>>> xk(0, 0)
______
```

```
>>> def how_big(x):
...     if x > 10:
...         print('huge')
...     elif x > 5:
...         return 'big'
...     elif x > 0:
...         print('small')
...     else:
...         print("nothin")
>>> how_big(7)
______

>>> how_big(12)
______

>>> how_big(1)
______

>>> how_big(-1)
______
```

```
>>> n = 3
>>> while n >= 0:
...     n -= 1
...     print(n)
------
```

```
>>> positive = 28
>>> while positive:
...    print("positive?")
...    positive -= 3
------
```

```
>>> positive = -9
>>> negative = -12
>>> while negative:
...    if positive:
...        print(negative)
...    positive += 3
...    negative += 3
------
```

## Q2: WWPD: Veritasiness

Use Ok to test your knowledge with the following "What Would Python Display?" questions:

```
python3 ok -q short-circuit -u
```

```
>>> True and 13
------

>>> False or 0
------

>>> not 10
------

>>> not None
------
```

```
>>> True and 1 / 0 and False
------

>>> True or 1 / 0 or False
------

>>> True and 0
------

>>> False or 1
------

>>> 1 and 3 and 6 and 10 and 15
------

>>> -1 and 1 > 0
------

>>> 0 or False or 2 or 1 / 0
------
```

```
>>> not 0
------

>>> (1 + 1) and 1
------

>>> 1/0 or True
------

>>> (True or False) and False
------
```

## Q3: Debugging Quiz!

The following is a quick quiz on different debugging techniques you should use in this class. You should refer to this document (/~cs61a/fa20/articles/debugging.html) to answer the questions!

Run the following to run the quiz.

```
python3 ok -q debugging-quiz -u
```

# Coding Practice

## Q4: Falling Factorial

Let's write a function `falling`, which is a "falling" factorial that takes two arguments, `n` and `k`, and returns the product of `k` consecutive numbers, starting from `n` and working downwards.

```
def falling(n, k):
    """Compute the falling factorial of n to depth k.

    >>> falling(6, 3)  # 6 * 5 * 4
    120
    >>> falling(4, 3)  # 4 * 3 * 2
    24
    >>> falling(4, 1)  # 4
    4
    >>> falling(4, 0)
    1
    """
    "*** YOUR CODE HERE ***"
```

Use Ok to test your code:

```
python3 ok -q falling
```

## Q5: Sum Digits

Write a function that takes in a nonnegative integer and sums its digits. (Using floor division and modulo might be helpful here!)

```
def sum_digits(y):
    """Sum all the digits of y.

    >>> sum_digits(10) # 1 + 0 = 1
    1
    >>> sum_digits(4224) # 4 + 2 + 2 + 4 = 12
    12
    >>> sum_digits(1234567890)
    45
    >>> a = sum_digits(123) # make sure that you are using return rather than print
    >>> a
    6
    """
    "*** YOUR CODE HERE ***"
```

Use Ok to test your code:

```
python3 ok -q sum_digits
```

# Submit

Make sure to submit this assignment by running:

```
python3 ok --submit
```

**Reminder**: Please fill out the Lab 0 survey (also included at the beginning of this assignment): this survey (https://forms.gle/BP3tcRgqPhHq4Ymf9)

# Extra Practice

> These questions are optional and will not affect your score on this assignment. However, they are **great practice** for future assignments, projects, and exams. Attempting these questions is valuable in helping cement your knowledge of course concepts, and it's fun!

## What Would Python Display? (Part 2)

### Q6: WWPD: What If?

> Use Ok to test your knowledge with the following "What Would Python Display?" questions:
>
> ```
> python3 ok -q if-statements -u
> ```
>
> **Hint**: `print` (unlike `return`) does *not* cause the function to exit!

```
>>> def ab(c, d):
...     if c > 5:
...         print(c)
...     elif c > 7:
...         print(d)
...     print('foo')
>>> ab(10, 20)
______
```

```
>>> def bake(cake, make):
...     if cake == 0:
...         cake = cake + 1
...         print(cake)
...     if cake == 1:
...         print(make)
...     else:
...         return cake
...     return make
>>> bake(0, 29)
------

>>> bake(1, "mashed potatoes")
------
```

# More Coding Practice

## Q7: Double Eights

Write a function that takes in a number and determines if the digits contain two adjacent 8s.

```
def double_eights(n):
    """Return true if n has two eights in a row.
    >>> double_eights(8)
    False
    >>> double_eights(88)
    True
    >>> double_eights(2882)
    True
    >>> double_eights(880088)
    True
    >>> double_eights(12345)
    False
    >>> double_eights(80808080)
    False
    """
    "*** YOUR CODE HERE ***"
```

Use Ok to test your code:

```
python3 ok -q double_eights
```

## CS 61A (/~cs61a/fa20/)

Weekly Schedule (/~cs61a/fa20/weekly.html)

Office Hours (/~cs61a/fa20/office-hours.html)

Staff (/~cs61a/fa20/staff.html)

## Resources (/~cs61a/fa20/resources.html)

Studying Guide (/~cs61a/fa20/articles/studying.html)

Debugging Guide (/~cs61a/fa20/articles/debugging.html)

Composition Guide (/~cs61a/fa20/articles/composition.html)

## Policies (/~cs61a/fa20/articles/about.html)

Assignments (/~cs61a/fa20/articles/about.html#assignments)

Exams (/~cs61a/fa20/articles/about.html#exams)

Grading (/~cs61a/fa20/articles/about.html#grading)